



ARTEMIS 小白指南

谷歌开源的"AI 操控真手机"框架——自然语言一句话，Agent 替你点、滑、测

github.com/google/artemis

★ 10.9k stars

Apache-2.0

Python 3.12+

LangGraph + LangChain

MCP 原生服务器

一句话：ARTEMIS 把自然语言指令翻译成可靠的 Android 真机操作，让 AI 助手（Claude Code / Antigravity / Codex...）通过 MCP 协议像真人一样操控手机——点按钮、跨 App 办事、跑测试、抓 Logcat，号称在谷歌官方 AndroidWorld 基准上拿到 99%+ 任务完成率。

本指南 = 12 个疑问逐条拆解 + 4 个"无真机也能跑"的真实实验（2026-10-03 全部实测通过）。

📖 疑问即目录（12 问）

1. ARTEMIS 到底是什么？一句话说明
2. 没有它，手机自动化是怎么做的？
3. 整体架构：AI 的"手"怎么伸进手机？
4. Flash 模式：3~5 秒一步的反应式循环
5. Pro 模式：七个节点的多 Agent 大图
6. 可靠性从哪来？多模态定位三层兜底
7. 截图历史爆上下文怎么办？双边缘压缩
8. Agent 手抖点错了怎么办？安全网与执行事件
9. MCP 接入 IDE：Claude Code 怎么"摸到"手机？
10. 两套模式怎么选？多设备并发怎么管？
11. 和我学过的概念怎么对上号？
12. 我能用它干什么？能力清单

🔗 逐条解答：从"是什么"到"怎么做到的"

Q1

ARTEMIS 到底是什么？一句话说明

ARTEMIS = 一个把"人话"翻译成"手机动作"的 AI 代理框架。你发一句 "打开设置，看看电量，然后放首 Coldplay"，它操纵一台真实的 Android 手机（或模拟器）逐步完成，全程像真人一样看屏幕、找按钮、点、滑、打字。

它同时是三样东西：① 一个手机 GUI Agent（看屏幕→思考→动手的循环体）；② 一个 MCP 服务器（把"操控手机"变成工具，供 Claude Code 等 IDE 调用）；③ 一个自动化测试引擎（自然语言写测试，替代脆弱的 UI 自动化脚本）。

谁在做"看和想"？多模态大模型（Gemini / Claude / GPT-4o / Qwen-VL 任选，路由器统一分发）；谁在做"动手"？ADB + uiautomator2 + 自家的 Accessibility Helper（一个装到手机上的无障碍服务 App）。**LLM 是大脑，ARTEMIS 是整套神经系统 + 手。**



Q2

没有它，手机自动化是怎么做的？

传统方案（Appium / Espresso / uiautomator 脚本）的套路是：人类先写死每一步——“找到 resource-id 为 `btn_next` 的按钮，点它”。能用，但有三个老大难：

痛点	传统脚本	ARTEMIS (AI Agent)
界面一改就崩	按钮那个位置 / 改个 id，脚本当场红	看的是屏幕语义，界面小改自动适应
跨 App 流程难写	每个 App 一套脚本，胶水代码巨多	一句自然语言描述目标即可
写脚本成本高	要会框架、会调试定位器	会说话就会“写测试”

代价也要诚实说：AI 每一步都要调大模型（Flash 约 3~5 秒/步，Pro 约 30 秒/轮）+ 模型 API 费；传统脚本执行是毫秒级且免费。所以官方定位很清楚——**ARTEMIS 负责“探索 + 生成 + 兜底”，最终交付物仍是可脱离 LLM 跑的确定性测试代码**（见它的 rules.md：“探索用 AI，交付可复现脚本”）。

Q3

整体架构：AI 的“手”怎么伸进手机？

仓库 608 个 Python 文件，主干是清晰的四层。每层只跟相邻层说话，所以底层驱动可以整个换成假的（我们的无真机实验就是这么做的）：

① 入口层

CLI (artemis run) · Web 控制台 (FastAPI :8000) · SDK · MCP 服务器 (mcp_server/, 给 IDE 当工具)

② Agent 层 (artemis/agents/, 15 个角色，共约 2 万行)

Flash 循环 · planner / operator / checker / validator / explorer / diagnoser / video_analyzer / log_analyzer ...
LangGraph 状态图把它们编成一张可控的图 (graph/, 7 节点带收敛闸门)

③ 动作/观察层

action_specs (12 个标准动作) · action_executor · observation (截图+XML 融合+元素索引) · llm/router (多供应商模型路由)

④ 驱动层 (drivers/) → 真机

AndroidAdbDriver (ADB+uiautomator2+Accessibility Helper) · CloudDriver · MockDeviceDriver (内存假手机，测试专用)

一句话记住：**Agent 层出“意图”，驱动层出“触觉”**——上层只说“点击第 3 个元素”，怎么让它发生（真机 ADB / 云手机 / 内存假手机）是驱动层的事。

Q4

Flash 模式：3~5 秒一步的反应式循环

Flash 是"单人快枪手"：**不进 LangGraph 大图，一个 LLM 驱动的观察-思考-行动循环转到底**。没有 Planner、没有 Checker，胜在快——官方给的节奏是每步 3~5 秒。



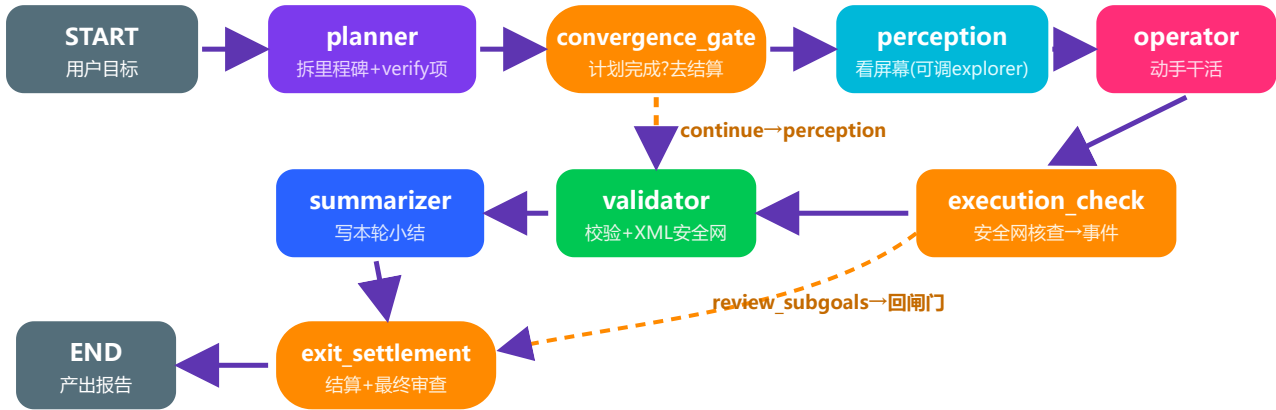
两个聪明细节 (源码 `agents/flash/runner.py` 实证)：

- ① **click_sequence 连击**：把多次点击打包成一个动作连发，抢在自动消失的播放条/Toast 消失前点完；
- ② **异步历史摘要**：旧步骤的截图在后台压缩成"视觉转变摘要"，主循环不等它，省时间又省上下文。

Q5

Pro 模式：七个节点的多 Agent 大图

Pro 是"正规军"：LangGraph 状态图编排七个节点，计划→执行→验证→总结闭环。下面这张拓扑是我们用脚本从 `graph/graph.py` 源码里提取的真实接线（另有收敛闸门 `convergence_gate` 由辅助函数接入）：



角色	一句话职责	代码规模(实测)
planner	把目标拆成"活计划": 里程碑 + verify/assert 检查项, 可动态改	436 行
operator	主力干活的: 看屏幕、点滑输、记笔记、必要时发 ADB 命令	2,458 行
validator	动作前安全网 (目标元素还在吗) + 执行校验	2,251 行
checker	只读验证员: 对着计划检查点做核查 + 退出终审	859 行
explorer	定位专家: flash/pro/ultra 三档 (1 发/3 轮/深缩放), operator 用 ask_explorer 问它	3,598 行
video_analyzer	看会话录像找线索 (全场最大的角色, 5,415 行)	5,415 行
summarizer / diagnoser / log_analyzer...	总结、诊断、日志分析等辅助角色	190~608 行

和 Flash 怎么分工? 官方 rules.md 写得很直白: 任务短且直选 Flash; 需要持续计划、检查点验证、日志/ADB 诊断、多分支探索, 或者要一份书面报告, 就上 Pro。

Q6

可靠性从哪来？多模态定位三层兜底

LLM 说"点设置图标"容易，**点得准**才是功夫。ARTEMIS 的定位是三层 fallback：



标准 App 走①②；自绘 UI（游戏/视频）才动用昂贵的③——便宜通道优先，贵的最后上

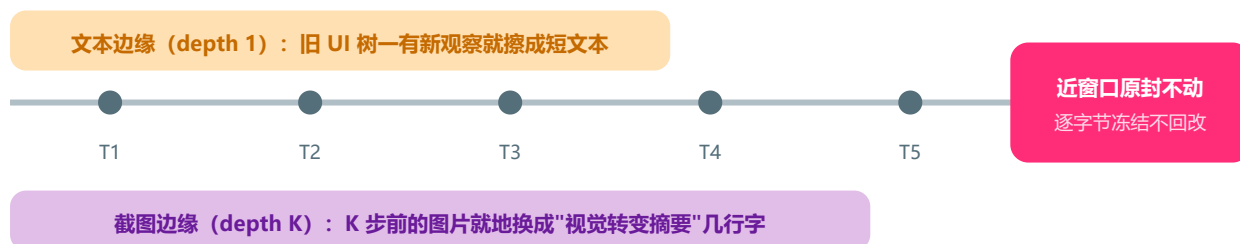
再加一道**"坐标自愈"**：动作落地前，validator 用 XML 层级核对目标元素"还在不在原处"（文本相似度 0.4 + 位置相似度 0.5 加权打分），小漂移直接修正坐标，真不见了才报失败，失败还分三类（shifted / occupied / disappeared）喂回给 Operator。

99%+ AndroidWorld 完成率怎么理解？ AndroidWorld 是 Google Research 的 116 个真机任务基准（跨 20 个真实 App）。99%+ 指 Agent 在该基准上的任务完成率——**这是"公开基准 + 可复现结果"级别的宣称，不是营销话术**；但注意它默认跑在指定模型（Gemini 系）上，换模型成绩会变。

Q7

截图历史爆上下文怎么办？双边缘压缩

长任务每步一张 1080×2400 截图 + 一坨 UI 树文本，几十步就把 200k 上下文撑爆。ARTEMIS 的答案是一个很优雅的机制：**ScrubEdgeCompressor 在历史上维持两条"橡皮擦边缘"**（源码 `agents/flash/context_compressor.py`）：



三条铁律（官方单测逐条验证过，我们实验 ex4 实跑 169 个相关测试全绿）：

- ① **冻结不变量**：滑出两条边缘的消息逐字节冻结，后续任何压缩都不回改——模型看到的不会"变卦"；
- ② **临时块即焚**：每轮的提醒/告警等临时消息，一出近窗口就被删干净，绝不混进长期历史；
- ③ **摘要只换图**：文字编辑与图片替换是两个独立边缘，race 时有宽限期，占位符永不回填假摘要。

这条和知识库 [\[\[Compaction上下文压缩算法\]\]](#)、[\[\[上下文预算与战略压缩\]\]](#) 是同一问题的三种答案：一个靠"切+摘要"，一个靠"提前压缩+预算"，ARTEMIS 靠"双边缘滑动擦除"。

Q8

Agent 手抖点错了怎么办？安全网与执行事件

Pro 模式给每个“收尾动作”上了**预执行安全网**：动作要碰的元素先在最新 UI 层级里验明正身，坐标小漂移就地自愈；同时允许多个快速动作打包成“**fast-action burst**”连发（跳过安全网抢时间）——这是打瞬逝 UI 的官方姿势。

动作被拦截或失败时，系统开一个**执行事件（execution incident）**：它带着连续失败计数**持久留在 Operator 的上下文里**，直到某个动作真正成功才自动销毁——修不修、怎么修由 Operator 自己决定，**没有独立的“修理工 Agent”，恢复是主力的责任**。

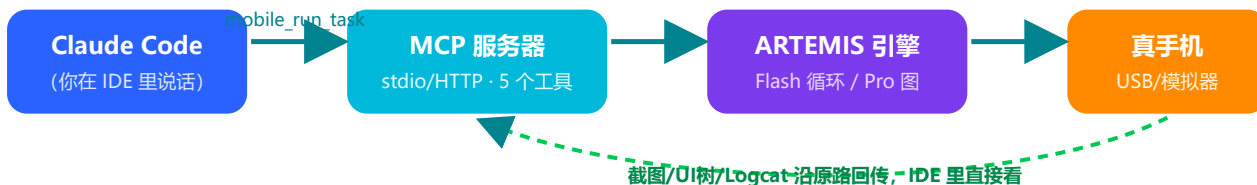


Q9

MCP 接入 IDE：Claude Code 怎么“摸到”手机？

ARTEMIS 内置一个原生 **MCP 服务器**（`mcp_server/`）。跑一次 `start.bat`，它会把 MCP 配置和一份《移动测试心法》（`rules.md`）自动装进检测到的 IDE。装好后，Claude Code 的工具箱里多出这些“手”：

MCP 工具	作用
<code>mobile_run_task</code>	派活：一条自然语言任务发给指定/自动选择的设备，后台子进程执行
<code>mobile_get_device_state</code>	看现场：实时截图或简化 UI 树（盯子代理进度、验证结果）
<code>mobile_diagnose</code>	诊断：抓 Logcat、分析失败原因
<code>mobile_inspect_trace</code>	复盘：查看执行轨迹（每步截图/动作/耗时）
<code>mobile_manage_tasks</code>	管理：列任务/停任务/多设备调度



这正是知识库 [\[\[MCP协议\]\]](#) 的教科书案例：没有 MCP，每个 IDE × 每个手机框架都要定制对接（M×N）；有了它，ARTEMIS 写一次服务器，所有支持 MCP 的 AI 助手即插即用（M+N）。[\[\[MCP服务器\]\]](#)、[\[\[工具调用生命周期\]\]](#) 在这个仓库里能看到生产级实现。

Q10

两套模式怎么选？多设备并发怎么管？

	⚡ Flash	🕒 Pro
机制	单 LLM 反应式循环，无图	LangGraph 7 节点闭环
节奏	约 3~5 秒/步	约 30 秒/轮（多角色接力）
擅长	短平快的确定性 UI 操作、抢瞬逝控件（连击）	长任务/多分支探索/连续监控/要书面报告
历史	同一个会话账本（T+mm:ss 会话时钟，截图折成摘要，旧步骤分块入库）	
验证	无独立 Checker	verification_level 四档：off / final（默认，只终审） / checkpoints / strict

多设备：每个任务可指定 `device_serial`，**不同设备的任务并发互不阻塞**；不指定就自动从设备池挑空闲机。规则也很“懂事”：检测到多台设备且意图不明时，要求 AI **必须先问用户选哪台**，不许自作主张。

Q11

和我学过的概念怎么对上号？

知识库概念	在 ARTEMIS 里的对应物
[[Agent循环]]	Flash 的 observe-think-act 循环就是它的手机版：错误回灌=动作失败结果进下一轮观察
[[LangGraph与Agent编排]] / [[共享状态与Reducer]]	Pro 图的 State: Pydantic 显式字段 + take_last / sticky_or 两种 Reducer, extra="forbid" 拒绝幽灵键
[[程序化工具调用]]	对偶视角：ARTEMIS 让模型逐动作“点菜”，而 click_sequence 是把多个原语打包一次执行——跨向程序化的中间形态
[[MCP协议]] / [[MCP服务器]]	mcp_server/ 把整个引擎包装成 5 个工具卖给所有 AI IDE
[[Compaction上下文压缩算法]]	ScrubEdgeCompressor 双边缘擦除：另一种压缩流派（滑动边缘 vs 切割点 + 模板摘要）
[[证据优先质检ProofOverClaims]]	rules.md 要求“先真机探索验证交互，再写测试代码”——不许凭假设写测试
[[接缝与桩实现StubSeam]]	MockDeviceDriver + FakeChatModel：官方自带的桩，让整个引擎无手机无 key 可跑（我们实验的靠山）

另有意外收获：README 第 322 行明确致谢 `minitap-ai/mobile-use`（Minitap, Inc., Apache-2.0），third_party/ 目录带 NOTICE/METADATA 署名文件——推文评论区“谷歌套皮不署名”的说法在仓库层面站不住脚，署名是规范做的（fork/复用上游且合规署名，正是开源的正确姿势）。

Q12

我能用它干什么？能力清单

结合小白到进阶的水平，五个现实用途：

- ① **手机上的“嘴替”**：一句“把相册里上周的截图整理进‘素材’文件夹”这类跨 App 琐事，交给 Flash 跑；
- ② **自然语言写 App 测试**：不用学 Appium，描述场景让 Agent 真机探索并产出可复跑的测试代码；
- ③ **给 Claude Code 装一只“手”**：MCP 接入后，写代码的 AI 能顺手在真机/模拟器上验证自己改的 App；
- ④ **稳定性/压测流水线**：Pro 模式跑长时探索测试，自动收集 Logcat、性能指标、录像证据并出报告；
- ⑤ **学 GUI Agent 架构的活教材**：多模态定位、上下文压缩、多 Agent 闭环、MCP 集成——每一块都是可以单独抄走的设计。

上手三件套：`git clone → start.bat`（自动装 ADB/scrcpy/FFmpeg/uv 依赖）→ 浏览器开 `localhost:8000` 有向导；CLI 单发任务 `uv run artemis run "打开设置查电量" --profile flash`。需要：一台开了 USB 调试的 Android 手机或模拟器 + 至少一家模型 API key（Gemini/Anthropic/OpenAI/Ollama 皆可）。



动手实验（无真机离线全跑通 · 2026-10-03 实测）

诚实声明：本机是 Windows 11，没有安卓真机/模拟器。因此实验走“仓库结构 + 核心逻辑 + 官方自带桩（MockDeviceDriver + FakeChatModel）”路线：跑的是 ARTEMIS 真实代码与真实测试，但**没有**在真手机上端到端点过屏幕；CLI 直跑 `artemis run` 在无设备时被设备发现环节拦下（“No Android device found”），也如实记录如下。

实验一：装环境 + 全量单测（99% 通过）

1 操作：`pip install uv` → `uv sync`（自动建 venv 装齐 LangGraph/LangChain/FastAPI/MCP 等全家桶，退出码 0）。目的：最小可分析环境。

2 操作：`set PYTHONUTF8=1` + `set ARTEMIS_FAKE_LLM=1`，然后

```
uv run pytest tests/unit -q --basetemp=F:/reminder/artemis/exercise/.pytest_tmp
```

目的：验证整库代码在本地真实可跑。

✓ 预期/实测：2473 passed / 18 failed / 8 skipped（99.0% 通过）。两个真实的坑：① 首跑 577 个 error，根因是 Windows 系统临时目录 `Temp\pytest-of-Windows` 拒绝访问（PermissionError WinError 5），用 `--basetemp` 指到本地目录即全清——环境问题不是代码问题；② 剩余 18 个失败集中在 CLI restart 与模型工厂实例化（无 API key 场景）。

实验二：亲手驱动“假手机” (exercise/ex2_mock_driver.py)

- 1 操作：实例化 `MockDeviceDriver(1080x2400)` 并 connect——一台内存里的 Android 假手机，出厂停在 Settings。
- 2 操作：依次下发 `tap(540,1200) → input_text("hello artemis") → swipe(上滑) → press_key(BACK) → launch_app`。
- 3 验证：读 `action_history` 流水账并逐条断言。

✓ 实测输出（节选）：五个动作回执全 True；流水账如实记录 `#1 tap{'x':540,'y':1200} ... #5 launch_app{'package':'com.android.settings'}`；`get_current_package()` 返回正确包名。踩坑记录：该方法是 async，忘了 `await` 首跑断言失败——修一行即过。这一步验证了“Agent 层出意图、驱动层出触觉”的接口契约真实成立。

实验三：源码级拓扑提取 (exercise/ex3_topology.py)

- 1 操作：正则扫 `graph/graph.py` 提取 `add_node/add_edge/add_conditional_edges`；扫 `mcp/action_specs.py` 提取动作名；统计 agents/ 各角色行数。

✓ 实测：Pro 图 7 节点 (planner/operator/validator/summarizer/execution_check/perception/exit_settlement) + 3 条固定边 + 1 个条件出口；设备动作集 12 个；角色行数榜 `video_analyzer 5415 行 (最大!)` > `explorer 3598` > `operator 2458` > `validator 2251` > `flash 2137`。结果存 `exercise/pro_graph.json`。意义：指南 Q5 的架构图不是照抄文档，是从源码里称出来的。

实验四：双边缘压缩器 169 测全绿

- 1 操作：

```
set ARTEMIS_FAKE_LLM=1 && uv run pytest tests/unit/agents/test_flash_scrub_edge.py
tests/unit/agents/test_flash_runner_ledger.py tests/unit/memory -q --basetemp=...
```

✓ 实测：169 passed / 0 failed (0.71 秒)。覆盖冻结不变量（历史消息逐字节不变）、竞态宽限（摘要没回来不许回填占位假货）、ephemeral 临时块出窗即焚、会话账本 T+mm:ss 时钟等核心机制。附带发现：同目录 `test_explorer.py` 有 13 个测试失败，根因是测试用 `MagicMock` 冒充 `provider`、撞上新加的严格枚举校验抛 `ValueError`——测试基建兼容性问题，非功能缺陷，如实记录。

🌟 一页总结

ARTEMIS 证明了“AI 操控真机”已经工程化到可以直接用的程度：Flash 用最简单的循环换速度，Pro 用多 Agent 图换可靠性；三层定位 + 安全网 + 执行事件把“模型会点错”这件事管理起来；双边缘压缩让长任务装得进上下文；MCP 让所有 AI IDE 一秒获得一只“机器手”。对学习者的最大价值：它是 GUI Agent 全套工程难题（定位/压缩/自愈/编排/集成）的一份开源参考答案。

查证记录：仓库 `google/artemis` 真实存在（Google 官方 org，10.9k★，Apache-2.0，2026-08-13 创建，持续更新）；README 明确署名复用 `minitap-ai/mobile-use`（Apache-2.0，third_party/ 含 NOTICE）；AndroidWorld 为 Google Research 公开基准。“谷歌开源”说法属实。本指南实验与数据均为 2026-10-03 本机实测，无真机部分已如实标注。